Annotated Example

The two `CREATE TABLE` statements below come directly from the Researcher–Lab ERD from lecture. Every line is labelled with the ERD concept it maps to.

```
-- ERD: entity Lab  ->  CREATE TABLE Lab
CREATE TABLE Lab (
  lab_id    INT            PRIMARY KEY,  -- key attribute (underlined in
     ERD)
  lab_name  VARCHAR(100)                 -- regular attribute
);

-- ERD: entity Researcher  ->  CREATE TABLE Researcher
CREATE TABLE Researcher (
  res_id  INT            PRIMARY KEY,    -- key attribute
  name    VARCHAR(100)   NOT NULL,       -- attribute, marked required
  email   VARCHAR(100),                  -- attribute, nullable by default
  lab_id  INT                            -- 1:N relationship: FK goes on N
     side
    REFERENCES Lab(lab_id)
    ON DELETE SET NULL                   -- if Lab row deleted, set lab_id
       to NULL
);
```

What this looks like as stored data

Lab		**Researcher**			
`lab_id`	`lab_name`	`res_id`	`name`	`email`	`lab_id`
1	VisionLab	1	Alice	alice@lab.ai	1
2	NLPLab	2	Bob	bob@lab.ai	2
		3	Carol	carol@lab.ai	1
		4	Dan	`NULL`	2

Dan has no email. ***`NULL`*** *is allowed because* ***`email`*** *has no* ***`NOT NULL`*** *constraint. Every* ***`lab_id`*** *in Researcher matches a row in Lab, the FK enforces this.*

DDL Quick Reference

Common data types

`INT`	integer
`SMALLINT`	small integer (–32k to 32k)
`BIGINT`	large integer
`REAL`	floating-point number
`NUMERIC`	exact decimal (use for money)
`CHAR(n)`	fixed string, always n chars
`VARCHAR(n)`	variable string, up to n chars
`DATE`	calendar date
`TIMESTAMP`	date and time

NULL and NOT NULL

no constraint	column is nullable by default
`NOT NULL`	rejects `NULL` on insert or update
`DEFAULT` *val*	used instead of `NULL` if omitted

`NULL` means "unknown", not zero, not empty string, not false.

PRIMARY KEY vs UNIQUE

PRIMARY KEY	one per table; cannot be `NULL`; uniquely identifies every row
UNIQUE	many per table; `NULL` is allowed (and ignored by the constraint)
UNIQUE NOT NULL	same as `UNIQUE` but disallows `NULL`

Syntax examples

```
-- inline single-column PK:
res_id  INT  PRIMARY KEY,

-- clause for multi-column PK:
PRIMARY KEY (bar, beer),

-- unique, nullable:
email  VARCHAR(100)  UNIQUE,

-- unique, required:
email  VARCHAR(100)
       UNIQUE NOT NULL,
```

Use Side A as a reference. Write actual SQL — not pseudocode. Boxes marked ***(discuss)*** *are debriefed with the class after individual work.*

Box 1 Spot the Errors *(4 min)*

The `CREATE TABLE` below has **four mistakes**. Circle each labelled line and write a one-line fix below it.

```
CREATE TABLE Dataset (
  name         VARCHAR,           -- (a) what is wrong here?
  size_tb      REAL,
  license      CHAR(20),
  uploaded_on  TIMESTAMP,
  PRIMARY KEY (size_tb),          -- (b) what is wrong here?
  researcher   VARCHAR(100)       -- (c) what is wrong here?
  UNIQUE NOT NULL                 -- (d) what is wrong here?
);
```

(a) fix: .....

(b) fix: .....

(c) fix: .....

(d) fix: .....

Box 2 Write From Scratch *(8 min)*

A research lab tracks **compute jobs**. Each job has: a unique job ID; the cluster it ran on (required); a runtime in hours (may be unknown); a cost in dollars (must be recorded).

Write the full `CREATE TABLE` statement. Use correct types and constraints, and add an inline comment on each column explaining your choice.

Box 3 PRIMARY KEY or UNIQUE? *(5 min, discuss)*

For each scenario choose one: `PRIMARY KEY` / `UNIQUE NOT NULL` / `UNIQUE` / *neither*. Write your choice and a one-sentence reason.

Scenario	Choice	Reason
`res_id` — auto-assigned integer, never null	.....	.....
`email` — must be recorded, no two researchers share one	.....	.....
`license` — e.g. CC-BY, ODC; may be unknown	.....	.....
`(model_id, dataset_id)` — no duplicate pairs allowed	.....	.....

Box 4 OK or Rejected? *(3 min, discuss)*

Each `INSERT` will either succeed or be rejected by Postgres. Circle **OK** or **REJ** and write one reason.

```
CREATE TABLE Model (
  model_id   INT          PRIMARY KEY,
  model_name VARCHAR(100) NOT NULL,
  version    CHAR(10)     UNIQUE,
  params_b   BIGINT
);
```

Statement	OK / REJ	Reason
`VALUES (1, 'GPT-X', 'v1.0', 7000000000)`	OK REJ	.....
`VALUES (1, 'LLaMA-4', 'v2.0', NULL)`	OK REJ	.....
`VALUES (2, NULL, /v2.0', 13000000000)`	OK REJ	.....
`VALUES (3, 'DiffNet', NULL, NULL)`	OK REJ	.....
`VALUES (4, 'DiffNet-2', NULL, NULL)`	OK REJ	.....

Rows 4 and 5 both have `NULL` for `version`. Does that violate `UNIQUE`?